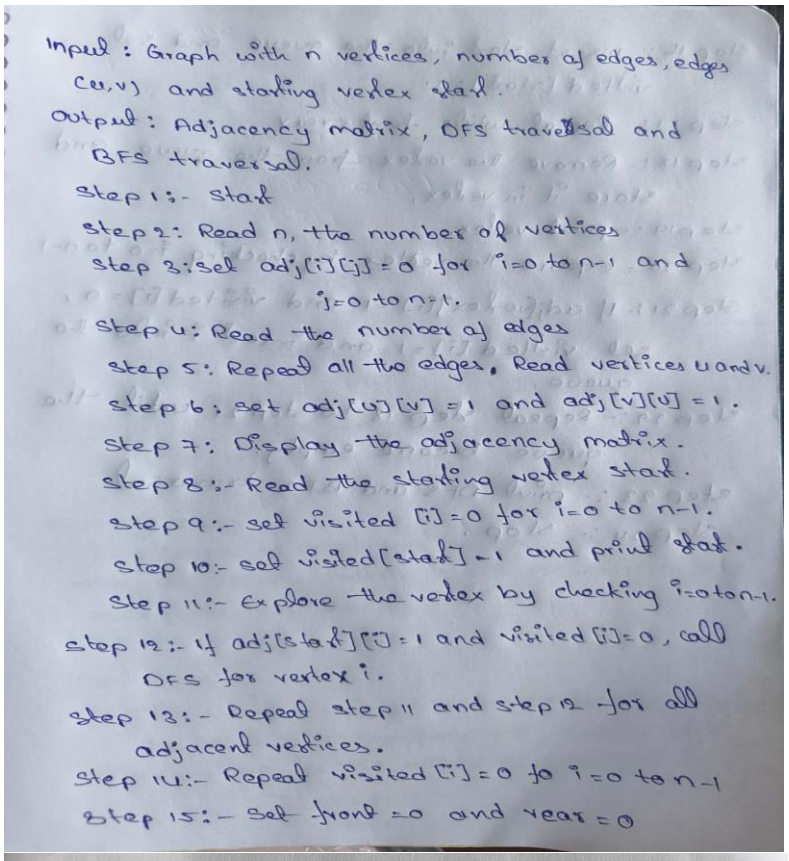
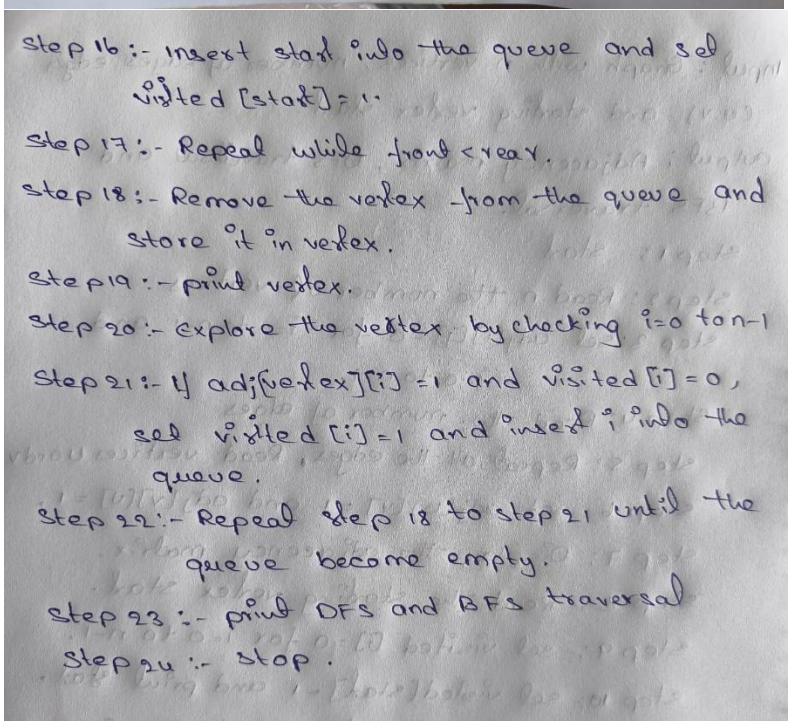
Practical- 8

Aim: To Implementation of Graph and Searching (DFS and BFS).

DFS and BFS

Algorithm:





Name: Chinni Thabitha
Enrollment Number:92460118038

Program:

```
#include <iostream>
using namespace std;
class Graph
{
public:
   int adj[10][10];
   int visited[10];
   int n;
   void getInput()
   {
      int edges;

      cout << "Enter number of vertices: ";
      cin >> n;
      for(int i = 0; i < n; i++)
      {
         for(int j = 0; j < n; j++)
         {
            adj[i][j] = 0;
         }
      }
      cout << "Enter number of edges: ";
      cin >> edges;
      cout << "Enter edges:\n";
      for(int i = 0; i < edges; i++)
      {
         int u, v;
         cin >> u >> v;
         adj[u][v] = 1;
         adj[v][u] = 1;
      }
   }
```

Name: Chinni Thabitha
Enrollment Number:92460118038

```
void displayGraph() {
   cout << "\nAdjacency Matrix:\n";
   for(int i = 0; i < n; i++){
      for(int j = 0; j < n; j++)
      {
         cout << adj[i][j] << " ";
      }
      cout << endl;
   }
}
void DFS(int start)
{
   visited[start] = 1;
   cout << start << " ";
   for(int i = 0; i < n; i++){
      if(adj[start][i] == 1 && visited[i] == 0) {
         DFS(i);
      }
   }
}
void BFS(int start)
{
   int queue[10];
   int front = 0;
   int rear = 0;
   for(int i = 0; i < n; i++) {
      visited[i] = 0;
   }
   visited[start] = 1;
   queue[rear] = start;
   rear++;
   while(front < rear)
   {
```

```
        int vertex = queue[front];
        front++;
        cout << vertex << " ";
        for(int i = 0; i < n; i++)
        {
            if(adj[vertex][i] == 1 && visited[i] == 0)
            {
                visited[i] = 1;
                queue[rear] = i;
                rear++;  }
        }
      }
   }
};
int main()
{
   Graph g;
   int start;
   g.getInput();
   g.displayGraph();
   cout << "\nEnter starting vertex: ";
   cin >> start;
   for(int i = 0; i < g.n; i++)
   {
      g.visited[i] = 0;
   }
   cout << "\nDFS Traversal: ";
   g.DFS(start);
   cout << "\nBFS Traversal: ";
   g.BFS(start);
   cout << endl;
   return 0;
}
```

Output:

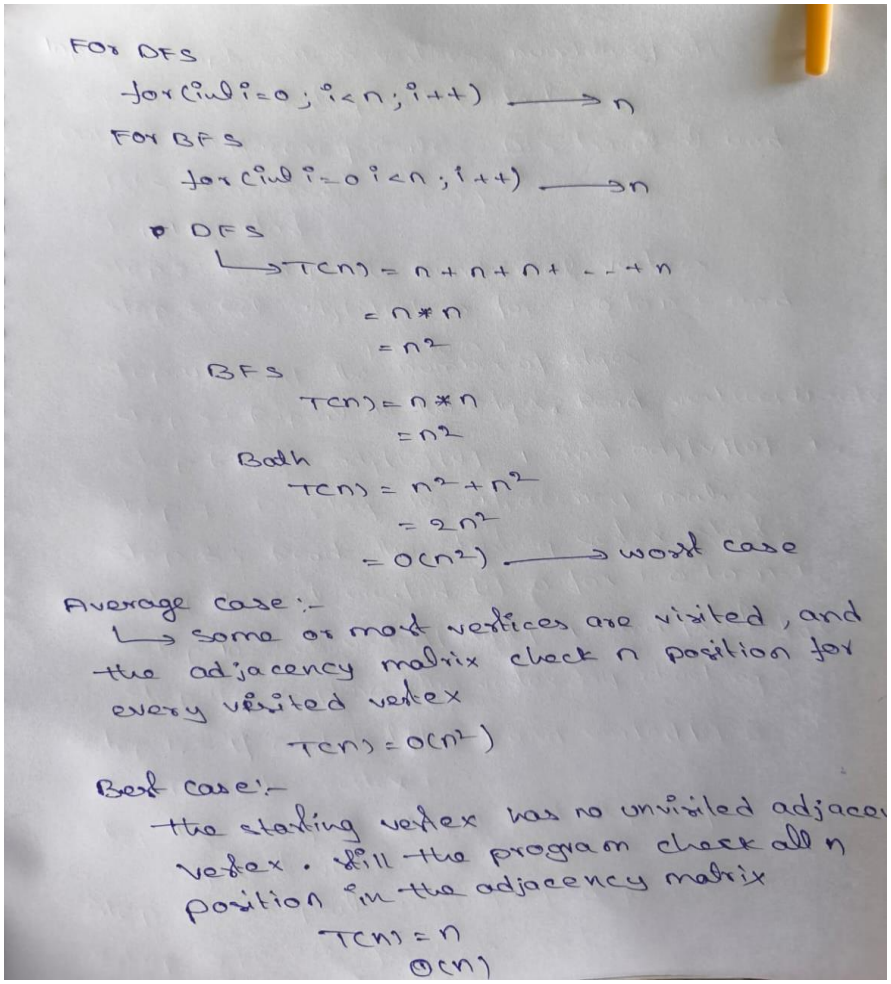
```
PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\practical-8\Bfs.exe"
Enter number of vertices: 6
Enter number of edges: 6
Enter edges:
0
1
0
2
1
3
1
4
2
5
4
5

Adjacency Matrix:
0 1 1 0 0 0
1 0 0 1 1 0
1 0 0 0 0 1
0 1 0 0 0 0
0 1 0 0 0 1
0 0 1 0 1 0

Enter starting vertex: 0

DFS Traversal: 0 1 3 4 5 2
BFS Traversal: 0 1 2 3 4 5
```

Time complexity:



Name: Chinni Thabitha
Enrollment Number:92460118038

Conclusion:

Thus, the Graph was successfully implemented using an adjacency matrix, and the DFS and BFS searching techniques were performed successfully. DFS explores the graph deeply using recursion, while BFS explores the graph level by level using a queue. The time complexity of the traversal is $O(n)$ in the best case and $O(n^2)$ in the average and worst cases for this implementation.

Name: Chinni Thabitha
Enrollment Number:92460118038